

SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE
FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL



PADRÕES DE UTILIZAÇÃO DO GITHUB

CLARISSON HENRIQUE DE PAIVA BENTO - 4005

MANOEL AUGUSTO ALVES DA COSTA - 5379

Florestal - MG

Setembro de 2025

Para o desenvolvimento do BitTint, optamos por um fluxo de *branches* simplificado, dividido em dois eixos principais: um para código, utilizando a *branch develop*, e outro para documentos, utilizando a *branch documents*. Essa escolha tem como objetivo garantir organização, rastreabilidade e controle de versões durante o projeto.

Branches

As *branches* utilizadas se dividem em fixas e temporárias. As fixas são *main*, *develop* e *documents*, enquanto as temporárias são *feature/** e *docs/**. As *branches* temporárias existem apenas enquanto a tarefa ou documento está em andamento e são fechadas após o *merge*.

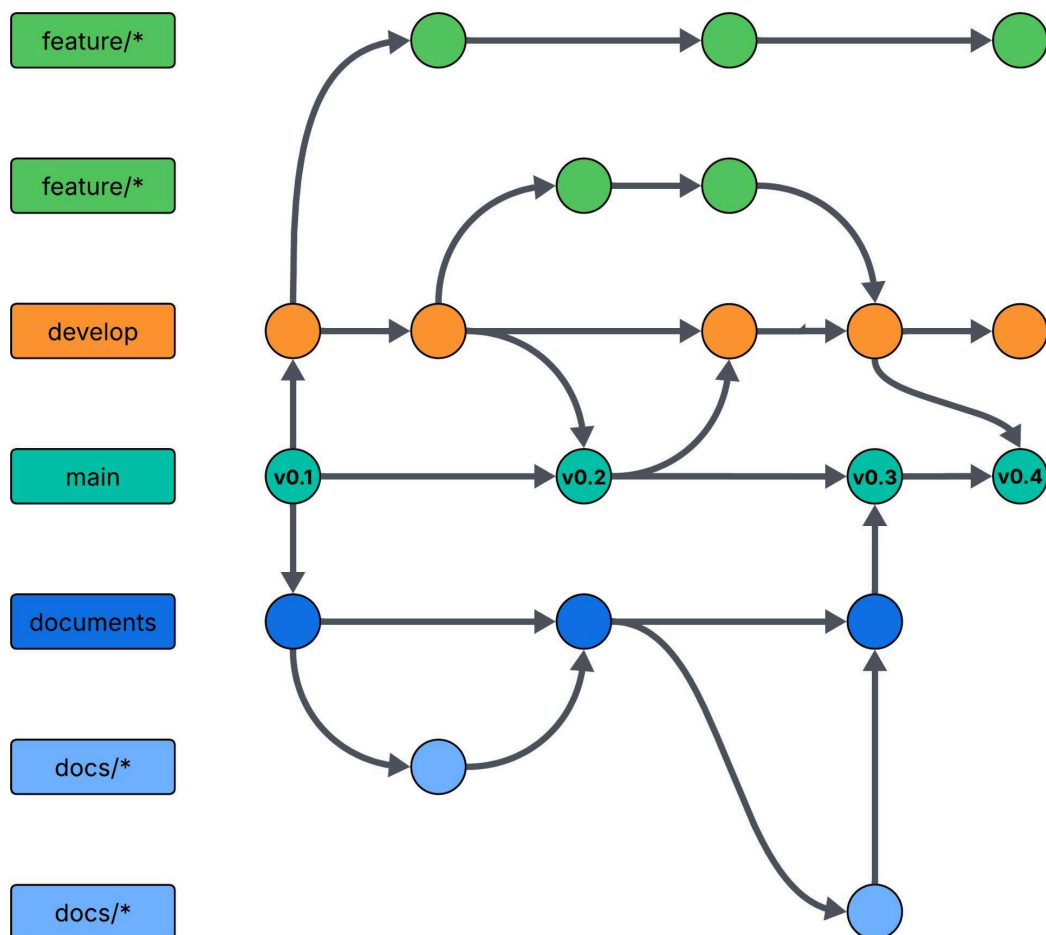


Figura 1: GitFlow para desenvolvimento do BitTint.

Branch Main

A *branch main* é a principal do repositório. Ela contém tanto o código quanto os documentos validados e serve como referência estável do projeto. Todo *merge* aceito na *main* deve receber uma *tag* de versão, como v0.1, para facilitar o rastreamento. Apenas as *branches develop* e *documents* podem interagir com a *main*, sempre no final da *sprint*. Diferente do *gitflow* tradicional, não utilizaremos *branches* de *hotfix*, portanto, *bugs* identificados na *main* só serão corrigidos na próxima versão.

Branch Develop

É a *branch* onde se mantém o código da próxima versão do projeto. Ela funciona como uma cópia da *branch* principal, mas com funcionalidades (*features*) em desenvolvimento que ainda não foram publicadas. Ao final da *sprint*, essas alterações são integradas à *branch main* por meio de um *merge*. Dessa forma, a *develop* serve como base para a criação e evolução de novas *features*.

Branch Documents

A *branch documents* tem função semelhante à *develop*, mas dedicada exclusivamente ao material de apoio e documentação do projeto. Nela ficam armazenados arquivos de texto, manuais, atas, relatórios, protótipos e quaisquer outros registros e artefatos relacionados a desenvolvimento. Assim como ocorre na *develop*, novas alterações são realizadas em *branches* derivadas e depois integradas à *documents* por meio de *merges*, garantindo que a documentação evolua de forma organizada e paralela ao código-fonte.

Merge/Pull Request

Ao concluir uma tarefa, seja ela um documento ou uma funcionalidade, o *merge* deve ser solicitado por meio de um *pull request*. Recomenda-se que o processo seja feito pela interface do GitHub, pois ela permite o uso de *labels* e *milestones*, ferramentas essenciais para a rastreabilidade do repositório. Além disso, utilize os campos *Reviewers* e *Assignees* para designar ambos os analistas de qualidade responsáveis pela revisão.

Após a abertura do *pull request*, o analista de qualidade analisará as alterações e decidirá, com base nos critérios estabelecidos, se a solicitação pode ser aprovada. Caso não atenda aos requisitos, o analista deve abrir uma *issue* detalhando os pontos que precisam ser corrigidos para que o *merge* seja aceito posteriormente.

É fundamental verificar a *branch* de destino correta antes de confirmar o *pull request*. Na tela

de criação, o campo “base:” representa a *branch* de destino (onde você deseja integrar suas alterações), enquanto o campo “compare:” representa a *branch* de origem (onde as alterações foram realizadas). Exemplos:

base: develop ← compare: feature/login/S

base: documents ← compare: feature/casos_de_uso/S1

Branch Feature

As *branches feature* são temporárias e auxiliares dentro do fluxo de desenvolvimento, utilizadas para implementar funcionalidades específicas. Adotaremos uma convenção de nomenclatura para todas as *branches* criadas ao longo do projeto:

feature/nome_do_recurso/SN, onde **S** representa a *sprint* e **N** o número da *sprint* correspondente.

No caso de alunos juniores, a nomenclatura deve seguir o padrão:

feature/nome_do_aluno/nome_do_recurso/SN.

Essas *branches* devem sempre ser criadas a partir da *branch develop*. Quando a funcionalidade estiver concluída e o *pull request* aprovado, a *branch feature* será removida. Se houver dez funcionalidades a serem desenvolvidas, devem ser criadas dez *branches* independentes. É importante destacar que as *branches feature* nunca interagem diretamente com a *main*, apenas com a *develop*.

Branch Docs

As *branches docs/** são temporárias e têm função análoga às *branches feature/**, mas voltadas exclusivamente para a produção e atualização de documentos do projeto. Elas devem sempre ser criadas a partir da *branch documents* e seguem uma convenção de nomenclatura:

docs/nome_do_documento/SN, em que **S** indica a *sprint* e **N** representa o número da *sprint*.

No caso de alunos juniores, deve-se utilizar o padrão:

docs/nome_sobrenome/nome_do_documento/SN.

Ao final da *sprint*, após a conclusão do documento e a aprovação do *pull request*, a *branch docs* será removida. Assim como nas *branches* de código, cada documento em desenvolvimento deve ter sua própria *branch* independente.

Padrão de Pull Request

No título do *pull request*, indique de forma resumida o propósito da *branch* e a funcionalidade ou documento a que se refere. Na descrição, informe de maneira geral o que foi realizado na *branch*, incluindo arquivos criados ou modificados e os trechos de código

afetados. Não é necessário detalhar as alterações realizadas, essa informação deve constar nos *commits*, que servem justamente para descrever o trabalho executado.

Padrão de Commit

Para realizar um *commit*, deve-se seguir o seguinte padrão de título para cada tipo:

- `:sparkles:` feat: título da feature (usado para novas features)
- `:books:` docs: título do documento (usado para novos documentos)
- `:bug:` fix: título da correção realizada (usado para correção de bugs)
- `:recycle:` refactor: título da refatoração (usado para refatorar códigos, ou seja, modificá-los sem alterar sua funcionalidade)
- `:star2:` style: título da estilização (usado para alterações que não afetam em nada o funcionamento e a performance do código)

Labels

Labels são tags usadas para categorizar *issues* e *pull requests*. As seguintes *labels* foram criadas no GitHub:

- ajuda necessária: precisa de suporte/discussão
- análise de qualidade: precisa de validação de qualidade
- bug: algo não está funcionando como deveria
- documento: correções ou alterações em documentos
- duplicado: essa issue ou pull request já existem
- feature: nova feature ou melhoria em uma feature existente
- fix: corrige um bug ou mal funcionamento de uma feature
- infraestrutura: CI/CD, build, config, etc...
- junior: Precisa da atenção do desenvolvedor sênior
- primeiro commit: tenha paciência com o junior

Milestones

As *Milestones* representam um tipo de tag também, mas essas servem para nomear cada *sprint* do projeto. Toda *issue* ou *pull request* deve estar vinculada à *Milestone* correspondente, permitindo assim o acompanhamento do progresso da *sprint*.

Dúvidas

Em caso de dúvidas sobre este documento ou sobre o fluxo do GitHub, deve-se entrar em contato pelo Discord do projeto, marcando o Gerente de Configuração, responsável por definir essas regras.